Granular ball local consensus for graph-free node classification under label noise

Silei Cao[a], Kun Wang[a], Jingyu Zhu[b], Tianxing Wang[c], Bing Huang[a,*]

[a]*School of Computer Science, Nanjing Audit University, Nanjing, 211815, PR China*
[b]*School of Statistics and Data Science, Nanjing Audit University, Nanjing, 211815, PR China*
[c]*School of Business, Nanjing Audit University, Nanjing, 211815, PR China*

Abstract

Graph-free node classification predicts from node attributes when graph relations are unavailable or costly at deployment. Existing noise-robust graph methods generally retain graph-dependent inference, whereas graph-free methods have largely been evaluated with clean training labels. We propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. GB-LCSS learns label-free graph contrastive representations, freezes them, and recursively groups nodes into granular balls. For each training node, labels of the other training nodes in its ball form a leave-one-out consensus whose agreement determines how strongly it is blended with the observed label to form a soft target for a multilayer perceptron (MLP). This separates region construction from noisy-label aggregation and reduces the influence of isolated annotation errors. Across five datasets and five noise settings, GB-LCSS achieves the highest five-seed mean accuracy in 24 of 25 settings, compared with seven noisy-label baselines sharing the same deployed MLP. Its soft targets also have lower mean Brier scores than the observed one-hot labels in all 25 settings.

Keywords: label noise, node classification, graph-free inference, graph contrastive learning, granular ball computing, soft labels

*Corresponding author.
Email address: huangbing@nau.edu.cn (Bing Huang)

1. Introduction

Node classification uses node attributes and graph relations for prediction. Graph neural networks (GNNs) combine them by aggregating neighboring representations [1], but this graph dependence complicates deployment because each prediction requires neighborhood access and message passing. Such access may be unavailable under missing relations, privacy restrictions, or low-latency requirements. Graph-free node classification instead uses graph structure during training while deploying an attribute-only classifier. Graph-MLP [2] and GLNN [3] established this setting, and later methods such as SimMLP [4] and FF-G2M [5] refined the transferred structural knowledge. These methods have largely been evaluated with clean training labels.

In practice, however, training labels obtained through manual annotation, automated rules, or multiple sources may contain errors, and deep models can fit such corrupted annotations [6]. On graphs, message passing may propagate the influence of mislabeled nodes to their neighbors [7]. Noise-robust graph methods use topology to augment supervision [8], regulate training with noisy labels [9], or select reliable samples [10], but they retain graph access at inference. Graph-free classifiers face a related challenge: errors in the labels used to train an MLP or its teacher can carry over to the deployed model. We study how the training graph can improve noisy-label supervision for a classifier that uses only node attributes at inference.

A natural strategy is to combine an observed label with evidence from nearby training nodes, so consistent local labels can offset isolated errors. Related methods use neighborhood consistency to identify mislabeled samples or encourage agreement among nearby predictions [11, 12]. For graphs, label-free contrastive representations provide a basis for identifying such local groups [13]. The key challenge is to form these regions before aggregating labels, preventing annotation errors from determining which nodes contribute evidence.

Granular ball computing provides such regions by grouping samples into coarse-grained units summarized by centers and radii [14]. Later work studied granular ball sampling for noisy or imbalanced classification [15] and adaptive ball generation [16].

In GB-LCSS, balls are formed from representation geometry under size and ball-count constraints before labels are aggregated.

Building on this idea, we propose Granular Ball Local-Consensus Soft Supervision (GB-LCSS) for transductive node classification with noisy labels and graph-free inference. As shown in Fig. 1, GB-LCSS first learns node representations through label-free graph contrastive learning and freezes them. It then recursively applies 2-means to the frozen node representations, assigning each node to exactly one granular ball. For each training node, the observed labels of the other training nodes in its ball provide a leave-one-out consensus. A weight determined by their agreement blends this consensus with the node's observed label to form a soft target. An MLP trained on these targets receives only node attributes at inference.

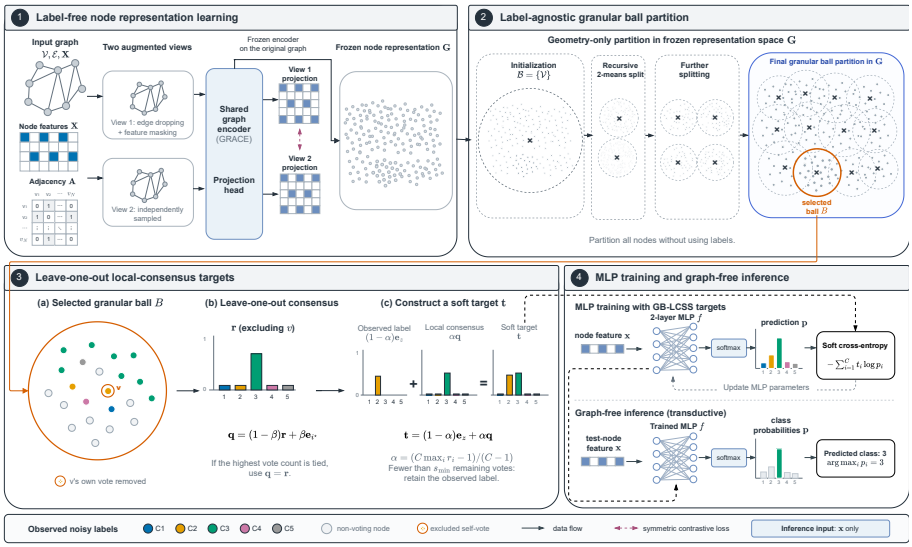


Figure 1: Overview of GB-LCSS. Stage 1 learns a label-free representation, and Stage 2 constructs granular balls in that representation space. Stage 3 shows an enlarged view of a representative ball to illustrate how a local-consensus target is formed. Stage 4 trains the deployed MLP with the resulting soft targets. Hollow nodes do not vote. Ball outlines are schematic, and each node belongs to exactly one ball. For clarity, node and ball indices are omitted in the schematic.

The main contributions of this paper are summarized as follows:

(1) We introduce GB-LCSS for noisy-label node classification with graph-free infer-

ence. It organizes nodes into granular balls in a frozen, label-free graph representation, thereby separating region construction from noisy-label aggregation.

(2) We construct node-specific soft targets by combining each observed label with the consensus of the other training nodes in its ball. Under independent symmetric label noise, we analyze how the number of votes and the underlying class distribution affect the reliability of local voting. We also derive an exact decomposition of the squared error of the soft target and a sufficient condition under which it has lower expected error than the observed label.

(3) We evaluate GB-LCSS on five datasets under five noise settings against noisy-label learning methods and graph-assisted MLP methods. Controlled comparisons examine grouping, representation learning, target construction, and parameter sensitivity. A graph-teacher extension evaluates the same targets against graph-to-MLP distillation methods.

2. Related work

2.1. Learning with noisy labels

Learning with noisy labels commonly relies on sample selection, consistency information, or robust objectives. Co-teaching [17] exchanges small-loss samples between two networks, while JoCoR [18] also encourages prediction agreement. Local relationships provide complementary evidence: neighbor-consistency regularization [11] encourages agreement among nearby samples, and affinity-aware uncertainty quantification (AUQ) [19] uses sample–prototype affinities to quantify uncertainty and refine pseudo-labels. Loss-based approaches include SCE [20], which combines cross-entropy with reverse cross-entropy, ELR [6], which limits noise memorization, and LogitClip [21], which constrains logit norms.

2.2. Noise-robust learning on graphs

Graph structure provides additional evidence for noisy-label learning, although message passing can also propagate supervision errors [7]. NRGNN [8] augments

supervision with structural information and pseudo-labeling, TSS [10] selects reliable nodes using topology, and RNCGLN [22] jointly addresses graph and label corruption. Recent studies further exploit reliability-aware graph information: MILAB [23] combines information-bottleneck-guided aggregation with adversarial consistency, while confidence-calibrated node selection [24] propagates annotation reliability using prototypes and graph structure. NoisyGL [7] provides a unified benchmark for graph learning under label noise. GB-LCSS uses the training graph to construct supervision for an attribute-only classifier deployed without graph access.

2.3. Graph-free node classification

Graph-free node classification uses graph information during training while avoiding online message passing at inference. Graph-MLP [2] adds a contrastive objective over graph neighborhoods, GLNN [3] distills a GNN teacher into an MLP, GSDN [25] uses neighborhood self-distillation, and SimMLP [4] aligns GNN and MLP representations without supervision. More recent distillation methods transfer different structural signals: FF-G2M [5] separates low- and high-frequency knowledge, KRD [26] estimates knowledge reliability under perturbations, and VQGraph [27] encodes local structures as discrete tokens.

Graph structure can also define soft targets. PosteL [28] uses neighborhood labels and graph-wide statistics, ALS [29] adaptively updates smoothed neighborhood targets, and SALS [30] incorporates structural information into the target distribution. GB-LCSS instead defines local regions in a frozen, label-free representation and excludes each node's own label from the consensus used to adjust it.

2.4. Graph contrastive representation learning

Self-supervised graph learning derives training signals from node attributes and graph connectivity [31]. Graph contrastive learning generates different views of a graph by perturbing its structure or attributes. A common node-level objective brings the representations of the same node across views closer together while distinguishing them from those of other nodes. The resulting node representations provide a basis for comparing nodes without using their observed class labels.

GRACE [13] implements this approach through random edge dropping and feature masking, with a shared graph encoder processing the two views. Because its objective uses no class labels, GB-LCSS can compute and freeze the GRACE representation before any training label is used; the observed training labels enter only afterwards, when soft targets are constructed within the resulting groups.

2.5. Granular ball computing

Granular ball computing represents samples with coarse-grained hyperspherical regions. GBkNN uses such regions for classification [14], granular ball sampling extends them to noisy or imbalanced data [15], and adaptive generation improves their construction [16]. MGNR [32] further builds multi-granularity neighborhood relationships from granular balls. More recently, BNO-GB [33] uses label-agnostic granular-ball clustering and multi-level consistency to identify noise types in open intent classification.

On graphs, supervised granular ball coarsening [34] and GBGC [35] construct coarsened graphs on which a GNN operates. Supervised classifiers and sampling methods may use label purity to guide ball generation [14–16]. Unlike BNO-GB's use of granular balls to identify noise types, GB-LCSS uses balls in a frozen, label-free graph representation to construct leave-one-out soft targets for an attribute-only node classifier. Ball membership determines which other training nodes contribute to each target.

3. Method

3.1. Problem setting and notation

We first define the learning task and notation summarized in Table 1.

Consider an attributed graph with node set $\mathcal{V} = \{v_1, \ldots, v_N\}$, edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$, adjacency matrix $\mathbf{A} \in \{0,1\}^{N \times N}$, and attribute matrix $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_N]^\top \in \mathbb{R}^{N \times d}$, where $\mathbf{x}_n$ is the attribute vector of node v_n.

Each node belongs to one of $C \geq 2$ classes, indexed by $i \in \{1, \ldots, C\}$. The set $\mathcal{V}_L \subseteq \mathcal{V}$ contains the labeled training nodes. Node v_n has a clean label y_n; if $v_n \in \mathcal{V}_L$, its observed label z_n may be corrupted. Both labels take values in $\{1, \ldots, C\}$. The

method uses observed labels, whereas clean labels serve as the reference for analysis and evaluation.

Indices n and ℓ denote nodes, j a granular ball, and i a class. Bold lowercase and uppercase letters denote vectors and matrices, respectively, and calligraphic letters denote sets or collections. A second subscript selects a class component.

Table 1: Main notation.

Symbol	Meaning
Graph and data	
$\mathcal{V}, \mathcal{E}$	Node set and edge set
N, C	Number of nodes and number of classes
n, ℓ	Node indices in $\{1, \ldots, N\}$
j	Granular ball index
i	Class index in $\{1, \ldots, C\}$
$\mathbf{A}, \mathbf{X}$	Adjacency matrix and node attribute matrix
$\mathbf{x}_n$	Attribute vector of node v_n
$\mathcal{V}_L$	Set of labeled training nodes
y_n, z_n	Clean and observed labels of node v_n
η	Label corruption probability
Representations and granular balls	
$\mathbf{G}, \mathbf{g}_n$	Frozen normalized GRACE representation matrix and vector for node v_n
d, h	Attribute dimension and GRACE representation dimension
$\mathcal{B}, B_j$	Collection of granular balls and its j-th member
$\boldsymbol{\mu}_j, D(B_j)$	Center and mean radius of ball B_j
$D'(B_j)$	Size-weighted mean radius after splitting B_j
$B_{\max}$	Maximum number of granular balls
$n_{\min}$	Minimum number of nodes in each accepted child ball
δ	Minimum relative decrease in mean radius for an accepted split
Local consensus and soft targets	
$\mathbf{w}_j$	Observed class-count vector of training nodes in B_j
U_n	Other training nodes in the ball containing v_n
$s_{\min}$	Minimum number of votes required to use local consensus
$\mathbf{r}_n$	Observed class proportions among the voting nodes in U_n
$i_n^\star$	Class with the unique highest vote count in U_n
$\mathbf{q}_n$	Local-consensus class distribution for node v_n
β	Strength of the adjustment toward the uniquely most-voted class
α_n	Weight of the local consensus in the target for v_n
$\mathbf{e}_i, \mathbb{I}(\cdot)$	One-hot vector for class i; indicator function
$\mathbf{t}_n$	Soft training target for node v_n
$f, \mathbf{p}_n$	MLP classifier and its predicted class distribution for v_n

During training, the full $\mathbf{X}$ and $\mathbf{A}$ are available for label-free representation learning and target construction, while test labels remain hidden. We learn an MLP f whose prediction for node v_n is

$$\mathbf{p}_n = \mathrm{softmax}(f(\mathbf{x}_n)). \tag{1}$$

Here, $p_{n,i}$ is the predicted probability that node v_n belongs to class i. After training, only f is retained for inference.

3.2. Label-free representation learning and granular ball construction

3.2.1. Label-free representation learning

We pretrain GRACE [13] on node attributes and graph connectivity without class labels, freeze the encoder, and normalize its pre-projection output for distance-based grouping. The resulting representation matrix is

$$\mathbf{G} = [\mathbf{g}_1, \ldots, \mathbf{g}_N]^\top \in \mathbb{R}^{N \times h}, \tag{2}$$

where $\mathbf{g}_n$ is the normalized representation of the n-th node and h is the representation dimension. Let $\mathbf{g}'_n$ denote the encoder output for node v_n before normalization. Its normalized representation is

$$\mathbf{g}_n = \frac{\mathbf{g}'_n}{\max(\|\mathbf{g}'_n\|_2, \varepsilon)}. \tag{3}$$

Here, $\|\cdot\|_2$ denotes the Euclidean norm, and $\varepsilon = 10^{-12}$ is a numerical safeguard. The same row-wise normalization is applied to all nodes to form $\mathbf{G}$.

Section 4.3.3 examines alternative representations and the variation across GRACE pretraining seeds.

3.2.2. Granular ball construction

Let $\mathcal{B} = \{B_1, \ldots, B_M\}$ be the partition of the node set formed in representation space, where $B_j \subseteq \mathcal{V}$ is the j-th granular ball and M is the final number of balls. Each node belongs to exactly one ball. Starting from $\mathcal{B} = \{\mathcal{V}\}$, the procedure recursively splits balls according to their representation-space dispersion.

Definition 1. For a nonempty granular ball B_j, each member node $v_n \in B_j$ has the normalized representation $\mathbf{g}_n$. The center $\boldsymbol{\mu}_j$ and mean radius $D(B_j)$ of the ball are

$$\boldsymbol{\mu}_j = \frac{1}{|B_j|} \sum_{v_n \in B_j} \mathbf{g}_n, \qquad D(B_j) = \frac{1}{|B_j|} \sum_{v_n \in B_j} \|\mathbf{g}_n - \boldsymbol{\mu}_j\|_2. \tag{4}$$

The mean radius is the average distance from member representations to the ball center.

Let $n_{\min}$ be the minimum number of nodes allowed in each child ball. A ball is eligible for splitting only if $|B_j| \geq 2n_{\min}$; eligible balls are examined in descending order of $D(B_j)$, and 2-means produces children $B_j^{(1)}$ and $B_j^{(2)}$.

Definition 2. The size-weighted post-split dispersion of B_j is

$$D'(B_j) = \frac{|B_j^{(1)}|D(B_j^{(1)}) + |B_j^{(2)}|D(B_j^{(2)})}{|B_j|}. \tag{5}$$

Let $\delta \in (0, 1)$ specify the minimum relative decrease in the mean radius. The split is accepted only if both child balls contain at least $n_{\min}$ nodes and

$$D'(B_j) \leq (1 - \delta)D(B_j). \tag{6}$$

The size weighting makes $D'(B_j)$ the average distance to the new centers, so Eq. (6) requires a relative decrease of at least δ. The first valid split is accepted and the candidates are reordered.

Splitting stops when no valid candidate remains or when the ball count reaches its limit $B_{\max}$, so that $M \leq B_{\max}$. Section 4.1.4 gives the experimental values of the partition parameters, and Algorithm 1 summarizes the procedure. Its Boolean flag accepted records whether a split has been accepted in the current pass; if no split is accepted, construction stops.

3.3. Local-consensus supervision and MLP training

Each training target combines the node's observed label with a consensus over the other training nodes in its ball.

3.3.1. Within-ball label consensus

For training node v_n in ball B_j, a leave-one-out vote excludes v_n itself.

Definition 3. The voting set U_n contains the other training nodes in B_j:

$$U_n = (B_j \cap \mathcal{V}_L) \setminus \{v_n\}. \tag{7}$$

Input: Normalized frozen representation $\mathbf{G}$, node set $\mathcal{V}$, parameters $B_{\max}, n_{\min}, \delta$

Output: Granular ball partition $\mathcal{B}$

$\mathcal{B} \leftarrow \{\mathcal{V}\}$;

while $|\mathcal{B}| < B_{\max}$ **do**

  accepted $\leftarrow$ false;

  foreach $B_j \in \mathcal{B}$ *with* $|B_j| \geq 2n_{\min}$*, in descending order of* $D(B_j)$ **do**

    Apply 2-means to the normalized representations of nodes in B_j to obtain $B_j^{(1)}$ and $B_j^{(2)}$;

    if $\min(|B_j^{(1)}|, |B_j^{(2)}|) \geq n_{\min}$ *and* $D'(B_j) \leq (1-\delta)D(B_j)$ **then**

      $\mathcal{B} \leftarrow (\mathcal{B} \setminus \{B_j\}) \cup \{B_j^{(1)}, B_j^{(2)}\}$;

      accepted $\leftarrow$ true;

      break;

    end

  end

  if accepted = false **then**

    break;

  end

end

return $\mathcal{B}$;

Algorithm 1: Recursive granular ball partitioning in the frozen representation space

Let $s_{\min} \geq 1$ be an integer specifying the minimum number of voting nodes required to use local consensus. When $|U_n| \geq s_{\min}$, the observed labels of the voting nodes define the empirical class distribution $\mathbf{r}_n = (r_{n,1}, \ldots, r_{n,C})^\top$, with

$$r_{n,i} = \frac{1}{|U_n|} \sum_{v_\ell \in U_n} \mathbb{I}(z_\ell = i), \qquad i = 1, \ldots, C. \tag{8}$$

Here, z_ℓ is the observed label of voting node v_ℓ, and $\mathbb{I}(z_\ell = i)$ equals 1 if this label is i and 0 otherwise. Excluding v_n prevents its observed label from reinforcing its own consensus.

Definition 4. If a single class receives more votes than every other class, its index is

$$i_n^\star = \arg\max_i r_{n,i}. \tag{9}$$

The local-consensus distribution $\mathbf{q}_n$ assigns additional weight to this class through the parameter $\beta \in [0, 1]$:

$$\mathbf{q}_n = \begin{cases} (1-\beta)\mathbf{r}_n + \beta \mathbf{e}_{i_n^\star}, & \text{if a unique highest-vote class exists} \\ \mathbf{r}_n, & \text{if the highest vote count is tied,} \end{cases} \tag{10}$$

Here, $\mathbf{e}_i$ is the one-hot vector with value 1 at class i and 0 elsewhere. The parameter β controls the weight assigned to the one-hot distribution of the uniquely most-voted class.

3.3.2. Consensus-weighted soft targets

The weight of $\mathbf{q}_n$ in the final target is determined by the largest class proportion in $\mathbf{r}_n$.

Definition 5. When $|U_n| \geq s_{\min}$, the consensus weight α_n is defined by rescaling the largest vote proportion relative to the uniform class probability $1/C$:

$$\alpha_n = \frac{C \max_i r_{n,i} - 1}{C - 1}. \tag{11}$$

Because $\mathbf{r}_n$ is a class distribution, its largest entry lies between $1/C$ and 1. Equation (11) maps this interval onto $[0, 1]$: uniformly distributed votes give $\alpha_n = 0$, and unanimous votes give $\alpha_n = 1$. For fixed votes, changing β leaves α_n unchanged.

Definition 6. The soft target $\mathbf{t}_n$ combines the observed label of v_n with its local consensus:

$$\mathbf{t}_n = \begin{cases} (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n \mathbf{q}_n, & |U_n| \geq s_{\min} \\ \mathbf{e}_{z_n}, & |U_n| < s_{\min}, \end{cases} \tag{12}$$

The target is a convex combination of the observed one-hot label and local consensus, with α_n controlling the consensus contribution.

Nodes with fewer than $s_{\min}$ other training nodes in their ball retain their observed one-hot labels.

Input: Granular ball partition $\mathcal{B}$, training nodes $\mathcal{V}_L$, observed labels $\{z_n : v_n \in \mathcal{V}_L\}$, number of classes C, parameters $s_{\min}, \beta$

Output: One stored soft target for each node in $\mathcal{V}_L$

Initialize $\mathbf{t}_n \leftarrow \mathbf{e}_{z_n}$ for every training node $v_n \in \mathcal{V}_L$;

foreach $B_j \in \mathcal{B}$ **do**

  $\mathbf{w}_j \leftarrow \sum_{v_\ell \in B_j \cap \mathcal{V}_L} \mathbf{e}_{z_\ell}$;

  foreach $v_n \in B_j \cap \mathcal{V}_L$ **do**

    $U_n \leftarrow (B_j \cap \mathcal{V}_L) \setminus \{v_n\}$;

    if $|U_n| < s_{\min}$ **then**

      continue;

    end

    $\mathbf{r}_n \leftarrow (\mathbf{w}_j - \mathbf{e}_{z_n})/|U_n|$;

    if *one class $i_n^\star$ has a strictly larger vote count than every other class* **then**

      $\mathbf{q}_n \leftarrow (1-\beta)\mathbf{r}_n + \beta \mathbf{e}_{i_n^\star}$;

    else

      $\mathbf{q}_n \leftarrow \mathbf{r}_n$;

    end

    $\alpha_n \leftarrow (C \max_i r_{n,i} - 1)/(C-1)$;

    $\mathbf{t}_n \leftarrow (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n \mathbf{q}_n$;

  end

end

return Stored targets for all nodes in $\mathcal{V}_L$;

Algorithm 2: Granular ball soft-target construction

Equations (7)–(12) use only the observed labels of training nodes; validation and test labels are not used in target construction. For each ball B_j, Algorithm 2 stores the observed class counts of its training nodes in a vector $\mathbf{w}_j$. For each training node $v_n \in B_j$, it subtracts the node's own vote from $\mathbf{w}_j$ and constructs $\mathbf{t}_n$ from the remaining counts.

3.3.3. MLP training

The two-layer MLP f consists of a hidden linear layer followed by ReLU, batch normalization, and dropout, and an output layer that produces class logits. It is trained by soft cross-entropy:

$$\mathcal{L} = -\frac{1}{|\mathcal{V}_L|} \sum_{v_n \in \mathcal{V}_L} \sum_{i=1}^{C} t_{n,i} \log p_{n,i}. \tag{13}$$

3.4. Analysis of local-consensus supervision

Under independent symmetric label noise, we bound local-vote error and derive a sufficient condition for lower expected soft-target error. The partition, training-node set, and clean labels are fixed; each label is retained with probability $1-\eta$ and otherwise replaced by a uniformly chosen incorrect class.

Fix a training node v_n with clean label y_n and $s_n = |U_n| \geq s_{\min}$ voting nodes. The clean class proportions and the margin of the true class over its closest competitor are

$$a_{n,i} = \frac{1}{s_n} \sum_{v_\ell \in U_n} \mathbb{I}(y_\ell = i), \qquad \Delta_n = a_{n,y_n} - \max_{i \neq y_n} a_{n,i}. \tag{14}$$

Here, $a_{n,i}$ is the clean class proportion in U_n; $\Delta_n > 0$ means the true class has more clean votes than any competitor. Probabilities are over the independent label corruptions.

Proposition 1. *Suppose that $C \geq 2$, $\Delta_n > 0$, and $0 \leq \eta < (C-1)/C$, and let $\gamma = 1 - C\eta/(C-1) > 0$. Each voting label is independently flipped with probability η to a uniformly chosen incorrect class. Then*

$$\Pr\left(\max_{i \neq y_n} r_{n,i} \geq r_{n,y_n}\right) \leq \min\left\{1, (C-1)\exp\left(-\frac{s_n \gamma^2 \Delta_n^2}{2}\right)\right\}. \tag{15}$$

Proof. Fix a competing class $i \neq y_n$. For each voting node $v_\ell \in U_n$, define $\xi_\ell = \mathbb{I}(z_\ell = y_n) - \mathbb{I}(z_\ell = i)$. These variables are independent and lie in $[-1, 1]$, and their average is $\bar{\xi} = r_{n,y_n} - r_{n,i}$. Under symmetric label corruption,

$$\mathbb{E}[\bar{\xi}] = \gamma(a_{n,y_n} - a_{n,i}) \geq \gamma \Delta_n.$$

Hoeffding's inequality [36] therefore gives

$$\Pr\left(r_{n,i} \geq r_{n,y_n}\right) = \Pr(\bar{\xi} \leq 0) \leq \exp\left(-\frac{s_n \gamma^2 \Delta_n^2}{2}\right).$$

Taking the union bound over the $C-1$ competing classes and capping the bound at one yields Eq. (15).

Symmetric corruption reduces the expected margin to $\gamma\Delta_n$, so more votes tighten the bound for fixed margin and noise rate. Larger groups may also reduce the margin, making both quantities relevant. Equation (10) preserves the most-voted classes, so the same bound applies to $\mathbf{q}_n$; Supplementary Appendix H.1 gives details.

Proposition 1 bounds the probability that local voting fails to give the true class a unique lead. We next assess the full soft-target distribution through its squared distance from the clean one-hot label. For node v_n, $\mathbf{e}_{z_n}$ is its observed one-hot label. The Brier score of a target vector $\mathbf{b}$ is $\mathrm{BS}_n(\mathbf{b}) = \|\mathbf{b} - \mathbf{e}_{y_n}\|_2^2$. The target $\mathbf{t}_n = (1-\alpha_n)\mathbf{e}_{z_n} + \alpha_n\mathbf{q}_n$ satisfies

$$\mathrm{BS}_n(\mathbf{t}_n) = (1-\alpha_n)\,\mathrm{BS}_n(\mathbf{e}_{z_n}) + \alpha_n\,\mathrm{BS}_n(\mathbf{q}_n) - \alpha_n(1-\alpha_n)\|\mathbf{e}_{z_n} - \mathbf{q}_n\|_2^2. \tag{16}$$

The identity is a weighted sum of the observed-label and consensus errors minus a nonnegative disagreement term. The following corollary gives a sufficient condition for improvement in expectation.

Corollary 1. *For the fixed node v_n with $|U_n| \geq s_{\min}$, assume independent symmetric label corruption with rate η. Fix the partition, the training-node set, the clean labels, and the observed labels of all other training nodes. Then $\mathbf{q}_n$ and α_n are fixed, and*

$$\mathbb{E}[\mathrm{BS}_n(\mathbf{t}_n)] \leq 2\eta(1-\alpha_n) + \alpha_n\,\mathrm{BS}_n(\mathbf{q}_n), \tag{17}$$

where the expectation is taken over the corruption of node v_n's label. If $\alpha_n > 0$ and $\mathrm{BS}_n(\mathbf{q}_n) < 2\eta$, the soft target has a lower expected squared error than the observed one-hot label.

Because node v_n is excluded from its own vote, $\mathbf{q}_n$ and α_n do not change with its observed label, whose one-hot vector has expected squared error 2η. Taking the expectation of Eq. (16) and omitting its nonpositive final term therefore gives Eq. (17) (Supplementary Appendix H.2). The condition compares the error of the consensus distribution with the expected error of a noisy annotation.

Across the training set, the change in Brier score combines the error reduction on corrupted labels with any error introduced on correct labels. Section 4.3.1 reports the net reduction, and Supplementary Appendix I.2 measures the two components separately.

3.5. Graph-teacher distillation extension

The base method trains the MLP on the labeled training nodes. In the extension, the same targets supervise a GCN teacher, whose predictions on all nodes are distilled into an MLP student.

The teacher is a two-layer GCN that takes the original attributes $\mathbf{X}$ and the adjacency matrix $\mathbf{A}$ as inputs. Its class-logit matrix is

$$\mathbf{O} = \mathrm{GCN}(\mathbf{X}, \mathbf{A}) \in \mathbb{R}^{N \times C}. \tag{18}$$

Let $\mathbf{o}_n \in \mathbb{R}^C$ denote the teacher-logit vector for node v_n, whose transpose is the n-th row of $\mathbf{O}$. The teacher loss is

$$\mathcal{L}_1 = -\frac{1}{|\mathcal{V}_L|} \sum_{v_n \in \mathcal{V}_L} \sum_{i=1}^{C} t_{n,i} \log[\mathrm{softmax}(\mathbf{o}_n)]_i. \tag{19}$$

The teacher therefore uses $\mathbf{G}$ only through the GB-LCSS targets; its inputs are $\mathbf{X}$ and $\mathbf{A}$.

After checkpoint selection, the teacher is frozen. With distillation temperature $\tau > 0$, its prediction for node v_n is

$$\boldsymbol{\pi}_n = \mathrm{softmax}\left(\frac{\mathbf{o}_n}{\tau}\right). \tag{20}$$

Here, $\pi_{n,i}$ is the teacher's probability for class i at node v_n. Increasing τ produces a softer distribution. The resulting distribution is stored as the distillation target of the node.

The MLP student f is trained on all N nodes, with these distributions as its sole distillation targets. Its loss is

$$\mathcal{L}_2 = -\frac{\tau^2}{N} \sum_{n=1}^{N} \sum_{i=1}^{C} \pi_{n,i} \log\left[\mathrm{softmax}\left(\frac{f(\mathbf{x}_n)}{\tau}\right)\right]_i. \tag{21}$$

After distillation, only the MLP student is retained for inference on node attributes.

3.6. Computational costs

Relative to direct MLP training, GB-LCSS adds three training-time stages: graph contrastive pretraining, granular ball partitioning, and soft-target construction. The pretraining cost depends on the encoder and the training schedule; Section 4.4.2 reports the measured cost. Pretraining is shared across downstream runs that use the same frozen representation.

The cost of partitioning is dominated by the 2-means calls. Running 2-means for I iterations on a ball B_j, with a fixed number of initializations, costs $O(I\,|B_j|\,h)$. The candidate balls examined in one pass of Algorithm 1 are disjoint, so their combined size is at most N. Each pass either accepts one split or terminates the algorithm. There are at most $B_{\max} - 1$ successful passes and one final unsuccessful pass, so the total cost of all 2-means calls is bounded by

$$O(I\,B_{\max}\,N\,h). \tag{22}$$

For fixed I and $B_{\max}$, this bound is linear in both the number of nodes and the representation dimension. Sorting the candidates by dispersion in each pass adds at most $O(B_{\max}^2 \log B_{\max})$ operations in total. The frozen representation $\mathbf{G}$ requires $O(Nh)$ space, and the node assignments, ball centers, and per-ball class counts require $O(N + B_{\max}(h + C))$ space. Given the per-ball class counts, computing the targets for the labeled nodes takes $O(|\mathcal{V}_L|C)$ time. The implementation initializes and stores a dense $N \times C$ target matrix, which adds $O(NC)$ time and space.

Let F be the MLP hidden width and T_S the number of epochs. The full-batch implementation evaluates all N nodes per epoch, for total cost $O(T_S N(dF + FC))$.
At inference, the cost per node is

$$O(dF + FC). \tag{23}$$

Model parameters require $O(dF + FC)$ space. The extension additionally trains a GCN teacher and an MLP student on all-node teacher predictions, while retaining the same deployed MLP and inference cost.

Table 2: Statistics of the five main datasets.

Dataset	Nodes	Undirected edges	Features	Classes	Training nodes	Edge homophily
Cora	2,708	5,278	1,433	7	541	0.810
CiteSeer	3,327	4,552	3,703	6	665	0.736
CoraFull	19,793	63,421	8,710	70	3,958	0.567
Coauthor-CS	18,333	81,894	6,805	15	3,666	0.808
Amazon-Photo	7,650	119,081	745	8	1,530	0.827

4. Experiments

4.1. Experimental setup

4.1.1. Datasets

The main experiments use five public datasets: the citation graphs Cora and CiteSeer [37], the 70-class citation graph CoraFull, the co-authorship graph Coauthor-CS, and the co-purchase graph Amazon-Photo [38]. Table 2 summarizes their statistics. Edges are counted as unique undirected pairs after self-loop removal, training-node counts refer to the 20% split, and edge homophily is the proportion of edges that connect nodes of the same class. Six additional datasets are evaluated under uniform 0.4 noise (Supplementary Appendix G).

4.1.2. Data splits and label noise

All comparisons follow the fixed-graph transductive protocol of Section 3.1. For each seed from 0 to 4, we use the NoisyGL percentage-split routine to randomly sample training, validation, and test nodes from the full node set at ratios of 20%, 10%, and 70%, respectively. Noise is injected only into the training labels, and all methods share the same split and noisy labels for each dataset and seed. Clean validation labels are used for checkpoint selection, and test labels for final evaluation. Supplementary Appendix F gives the exact subset sizes and the sampling procedure, and Supplementary Appendix C examines sensitivity to corrupted validation labels.

Under uniform noise, each training label is retained with probability $1 - \eta$ and is replaced by each of the other $C - 1$ classes with probability $\eta/(C - 1)$. We evaluate $\eta \in \{0.2, 0.4, 0.6\}$. Pair noise uses $\eta = 0.4$: it changes class $i \in \{1, \ldots, C\}$ to $((i-2) \bmod C)+1$ with probability η and leaves it unchanged with probability $1-\eta$. Instance-dependent

noise is generated with the NoisyGL generator, using a configured mean flip rate of 0.4 and norm_std = 0.1.

4.1.3. Compared methods

We compare with seven noisy-label baselines: MLP+CE, Small-loss, Co-teaching, JoCoR, SCE, ELR, and LogitClip. All use the same deployed two-layer MLP and labeled training nodes as GB-LCSS. Small-loss, Co-teaching, and JoCoR use the true noise rate for their forget-rate schedules; the latter two train independent MLP pairs but evaluate one network without ensembling. GB-LCSS requires no noise-model parameters.

The graph-assisted MLP methods are PosteL [28], ALS [29], SALS [30], Graph-MLP [2], and GSDN [25], all using graph information only during training in our protocol. PosteL uses the official target-construction code with iterative pseudo-labeling disabled; Graph-MLP uses the common MLP, while ALS, SALS, and GSDN are mechanism adaptations. Supplementary Appendix F details retained components and parameter sources.

Graph-to-MLP comparisons include a GCN(X)+CE response-distillation baseline based on GLNN [3], FF-G2M [5], KRD [26], VQGraph [27], and a GCN teacher trained on GB-LCSS targets (Section 3.5). FF-G2M is reimplemented with the common backbones and its frequency-specific objectives; mechanism adaptations of KRD and VQGraph retain reliability-based selection and soft-codebook distillation, respectively. Each route retrains teacher and student per seed with the same student MLP.

4.1.4. Implementation details

Datasets are loaded through the NoisyGL interface to PyTorch Geometric. Graph contrastive pretraining uses row-normalized attributes, whereas downstream models use the attributes supplied by NoisyGL. Graph-convolutional models use the supplied adjacency, with normalization and self-loops handled within their graph-convolution layers.

The noisy-label baselines and GB-LCSS use a two-layer MLP with a hidden width of 64 and a dropout rate of 0.5. We train for 200 epochs using Adam with a learning

rate of 0.01 and a weight decay of 5×10^{-4}, and we select the checkpoint with the highest validation accuracy. Teacher checkpoints are selected with the same criterion.

By default, GB-LCSS uses $B_{\max} = 64$, $n_{\min} = 32$, $\delta = 0.05$, $s_{\min} = 2$, and $\beta = 0.5$. Recursive 2-means uses Euclidean distance, k-means++ initialization [39], and 10 initializations per candidate split, with randomization controlled by the downstream seed. The distillation extension uses a temperature of $\tau = 1$. GRACE configurations and all other parameter settings, with their sources, are given in Supplementary Appendix F.

4.1.5. Evaluation protocol

Test accuracy is reported as mean ± sample standard deviation over five paired seeds. Bold and underline mark the highest and second-highest means, respectively, where applicable. Each dataset uses one fixed normalized pretrained representation $\mathbf{G}$; Section 4.3.3 examines variation across GRACE pretraining seeds.

Same-seed accuracy differences are assessed with two-sided paired t-tests and 95% confidence intervals. Holm correction [40] is applied separately to each family of 25 tests, with one family for each reference method: random size-matched grouping, MLP+CE, and JoCoR. Supplementary Appendix B reports the paired comparisons, confidence intervals, and Holm-adjusted p-values.

4.2. Comparative results

4.2.1. Comparison with noisy-label baselines

GB-LCSS attains the highest mean accuracy on all five datasets under uniform 0.4, exceeding the strongest baseline on each dataset by 1.33–5.54 percentage points (Table 3).

Across all five noise settings, it achieves the highest mean in 24 of the 25 settings, with average gains of 7.74 points over MLP+CE and 3.46 points over JoCoR, the strongest baseline on average across the 25 settings (Fig. 2). The only exception is Coauthor-CS under uniform 0.2, where JoCoR is higher by 0.27 points. Supplementary Appendix A gives the complete accuracy results, and Supplementary Appendix B reports the paired statistical comparisons.

Table 3: Test accuracy of noisy-label methods with the same deployed MLP under uniform 0.4 noise.

Method	Cora	CiteSeer	CoraFull	Coauthor-CS	Amazon-Photo
MLP+CE	44.24±2.62	43.50±3.02	31.98±0.98	88.42±0.62	73.90±1.80
Small-loss	51.95±1.97	49.70±3.07	33.04±0.50	89.06±0.88	77.25±1.62
Co-teaching	46.83±1.78	46.24±2.89	34.39±0.24	88.80±0.97	78.21±1.00
JoCoR	51.72±1.30	49.91±3.37	35.70±0.32	89.38±0.61	79.57±0.53
SCE	43.60±1.86	42.09±3.47	29.29±0.45	88.37±0.53	76.08±3.11
ELR	44.28±1.38	43.17±3.95	32.14±0.67	88.29±0.73	73.93±2.12
LogitClip	47.50±1.50	46.57±3.52	34.69±1.17	88.40±0.21	74.98±2.38
GB-LCSS	**57.49±0.76**	**55.24±2.04**	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

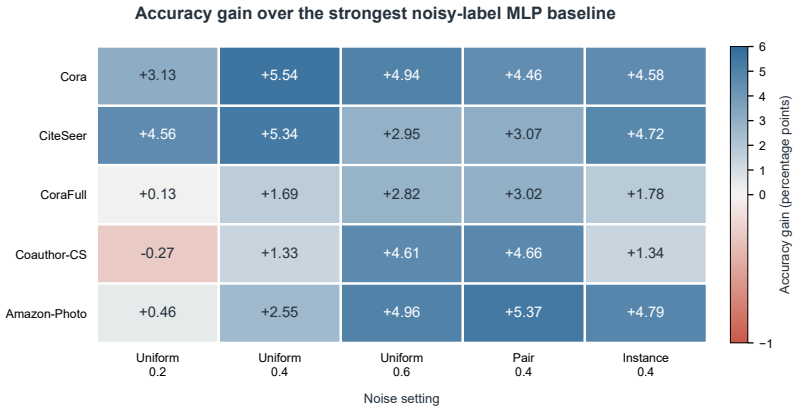


Figure 2: Accuracy gain of GB-LCSS over the strongest noisy-label baseline sharing the deployed MLP. Each cell is the five-seed mean difference for one dataset–noise setting, in percentage points; blue and red indicate positive and negative gains.

Fig. 3 shows accuracy as a function of the uniform noise rate. As the rate increases from 0.2 to 0.6, GB-LCSS loses less accuracy than MLP+CE and JoCoR on all datasets except CiteSeer.

On six additional datasets under uniform 0.4 noise, GB-LCSS exceeds random size-matched grouping on all six and the strongest noisy-label baseline on three (Supplementary Appendix G).

4.2.2. Comparison with graph-assisted MLP methods

Table 4 compares the graph-assisted MLP methods.

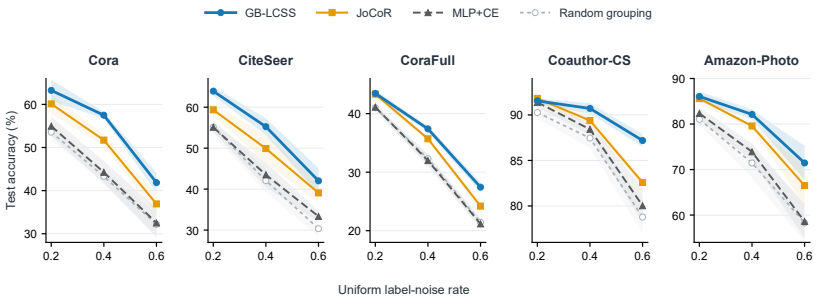


Figure 3: Test accuracy under uniform label noise. Lines and shading show five-seed means and sample standard deviations; panels use different y-axis scales. "Random grouping" preserves granular-ball counts and sizes.

Table 4: Test accuracy of methods that use the graph during MLP training, under uniform 0.4 noise.

Method	**Use of graph structure in training**	**Cora**	**CiteSeer**	**CoraFull**	**Coauthor-CS**	**Amazon-Photo**
PosteL	Posterior soft labels	47.09±1.81	45.84±2.48	36.51±1.01	84.74±1.51	80.59±0.94
ALS	Neighborhood label smoothing	45.20±1.24	43.77±2.76	31.91±1.05	88.55±0.86	73.41±2.75
SALS	Structure-aware label smoothing	48.27±2.50	47.20±1.69	37.06±0.78	88.85±0.82	77.86±0.79
Graph-MLP	Multi-hop neighborhood-contrastive objective	**63.23±3.07**	**57.23±1.85**	37.05±0.91	88.95±0.43	77.05±1.24
GSDN	Neighborhood self-distillation	55.97±4.29	50.36±3.39	35.80±0.66	86.55±0.78	73.31±2.00
GB-LCSS	Label consensus within granular balls	57.49±0.76	55.24±2.04	**37.39±0.51**	**90.71±0.59**	**82.12±1.08**

Across the 25 settings, the average accuracy gains over PosteL, ALS, and SALS are 5.69, 7.71, and 4.69 percentage points, respectively. GB-LCSS achieves higher mean accuracy than PosteL and SALS in 24 settings each, and than ALS in all 25 settings (Supplementary Appendix E). These methods use graph-defined neighborhoods for target construction, whereas GB-LCSS uses regions of a label-free representation and leave-one-out voting.

Graph-MLP and GSDN use the graph to shape the representation of the MLP and leave the labels unchanged. Under uniform 0.4, GB-LCSS has a higher mean than Graph-MLP on CoraFull, Coauthor-CS, and Amazon-Photo, by 0.34, 1.76, and 5.07 points, respectively, whereas Graph-MLP has the higher mean on Cora and CiteSeer, by 5.74 and 1.99 points. GB-LCSS exceeds GSDN on all five datasets. Graph-MLP

Table 5: Target-construction ablations under uniform 0.4 noise.

Dataset	GB-LCSS	Own vote included	Random size-matched grouping	Uniform smoothing	Prior smoothing	Gain from excluding own vote
Cora	**57.49±0.76**	56.28±1.54	43.31±1.40	44.69±2.90	44.89±2.38	+1.21
CiteSeer	**55.24±2.04**	53.52±2.71	42.08±2.88	43.69±3.31	43.43±2.99	+1.72
CoraFull	**37.39±0.51**	37.30±0.57	32.30±0.85	32.59±1.12	32.63±0.84	+0.09
Coauthor-CS	**90.71±0.59**	90.62±0.47	87.46±1.07	88.25±0.80	88.04±0.73	+0.09
Amazon-Photo	**82.12±1.08**	81.33±0.70	71.45±3.29	74.18±1.72	74.72±1.44	+0.79

and GB-LCSS thus use the training graph in different ways, shaping the learned representation and refining the supervision targets, respectively, and neither is uniformly better across datasets.

4.3. Ablation and mechanism analysis

4.3.1. Effect of local-consensus targets

We examine whether gains depend on which nodes contribute to local consensus. Random size-matched grouping shuffles membership while preserving ball sizes and the target rule. Uniform and prior smoothing mix each observed one-hot label with a uniform distribution or training-label frequencies using coefficient 0.1. Another variant includes the current node's own label in the vote under the same tie-handling rule. Table 5 reports results under uniform 0.4.

Random size-matched grouping lowers accuracy in all 25 settings, by 8.69 percentage points on average, to approximately the level of MLP+CE. Uniform and prior smoothing recover little of this gain. These controls support the importance of which nodes contribute to the local consensus, beyond target softening alone. Excluding the current node's vote raises mean accuracy on all five datasets under uniform 0.4, as well as on every dataset when averaged over the five noise settings. With clean training labels, targets change only in balls whose training nodes belong to more than one class. Accuracy differs from that of MLP+CE by -1.75 to $+2.00$ percentage points (Supplementary Appendix D).

We also assess the targets independently of the classifier. Fig. 4 reports their multiclass Brier score [41], averaged over labeled training nodes against clean labels used

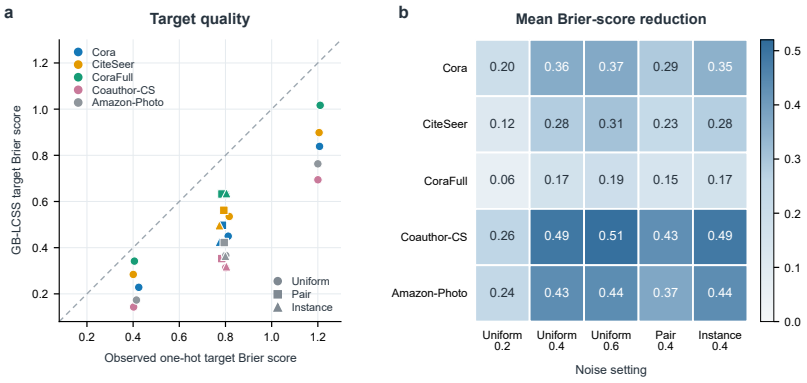


Figure 4: Brier scores against clean training labels. (a) Observed one-hot targets versus GB-LCSS targets; markers are five-seed means for dataset–noise settings, with shape indicating noise type. The dashed line is equality. (b) Observed-target minus GB-LCSS score; positive values favor the soft targets.

only for evaluation:

$$\frac{1}{|\mathcal{V}_L|} \sum_{v_n \in \mathcal{V}_L} \|\mathbf{t}_n - \mathbf{e}_{y_n}\|_2^2.$$

The score of the observed labels is computed by replacing $\mathbf{t}_n$ with $\mathbf{e}_{z_n}$. All 25 points, each averaged over five seeds, lie below the equality line: the GB-LCSS targets reduce the score by 0.06–0.51 relative to the observed one-hot targets, with the largest reductions on Coauthor-CS and Amazon-Photo.

Using Eq. (H.5), Supplementary Appendix I.2 separates the net Brier-score reduction under uniform 0.4 noise into error removed on corrupted labels and error introduced on correct labels. The reduction exceeds the increase in all 25 dataset–seed runs, by a factor of 6.7–18.1 in the five-seed means.

4.3.2. Sensitivity to the grouping rule

To test dependence on the partitioning rule, we keep the frozen representation and Eqs. (7)–(12) fixed and replace granular balls with kNN or k-means. kNN excludes the query node and matches the mean training-vote count per ball; k-means uses the same number of groups and also excludes the current node's vote. Table 6 reports uniform-0.4 accuracy differences relative to granular balls.

Table 6: Sensitivity to the grouping rule under uniform 0.4 noise.

Dataset	Granular ball	kNN	kNN − GB	k-means	k-means − GB
Cora	**57.49±0.76**	57.21±1.19	−0.27	56.58±1.23	−0.91
CiteSeer	55.24±2.04	56.17±3.24	+0.94	**56.18±1.78**	+0.94
CoraFull	37.39±0.51	**40.50±0.51**	+3.11	39.26±0.48	+1.87
Coauthor-CS	90.71±0.59	90.83±0.57	+0.12	**91.01±0.64**	+0.29
Amazon-Photo	82.12±1.08	**82.23±1.20**	+0.11	81.68±0.66	−0.44

GB denotes granular ball grouping. Differences are in percentage points.

Table 7: Effect of the representation used for partitioning under uniform 0.4 noise.

Dataset	Accuracy			Number of balls		
	GRACE	Raw $\mathbf{X}$	$\mathbf{A}^2\mathbf{X}$	GRACE	Raw $\mathbf{X}$	$\mathbf{A}^2\mathbf{X}$
Cora	**57.49±0.76**	44.37±2.31	44.37±2.31	52–53	1	1
CiteSeer	**55.24±2.04**	42.90±2.50	42.90±2.50	63–64	1	1
CoraFull	**37.39±0.51**	31.86±1.05	31.86±1.05	64	1	1
Coauthor-CS	**90.71±0.59**	87.63±1.10	89.20±0.67	64	1	50–59
Amazon-Photo	**82.12±1.08**	73.37±1.13	81.13±1.64	64	1	64

The three grouping rules yield mean accuracies within one percentage point of one another on four of the five datasets (Table 6). All three exceed random size-matched grouping (Tables 5 and 6). These results show that local-consensus targets can also be constructed with other grouping rules. We use granular balls because geometric splitting adapts the partition under explicit size and ball-count constraints, while each ball provides shared class counts for constructing its members' targets. Section 4.4.2 examines the construction time and storage requirements of this design.

4.3.3. Effect of the representation space

Table 7 compares grouping with GRACE, raw attributes $\mathbf{X}$, and two-step propagated attributes $\mathbf{A}^2\mathbf{X}$ under uniform 0.4 noise, using five paired seeds and fixed splitting parameters. Ball counts are reported as ranges across these seeds. For $\mathbf{A}^2\mathbf{X}$, we multiply twice by the original adjacency without added self-loops or normalization, then normalize the resulting vectors as in Eq. (3).

With raw $\mathbf{X}$, no split satisfies Eq. (6) in any of the 25 dataset–seed runs, so all nodes remain in one ball and each node draws its consensus from all other training nodes. The

$\mathbf{A}^2\mathbf{X}$ representation likewise produces a single ball on Cora, CiteSeer, and CoraFull. It produces 50–59 and 64 balls on Coauthor-CS and Amazon-Photo, respectively, where its accuracy exceeds that of raw $\mathbf{X}$. GRACE yields a multi-ball partition on every dataset and the highest mean accuracy among the three representations, exceeding $\mathbf{A}^2\mathbf{X}$ by 1.51 points on Coauthor-CS and 0.99 points on Amazon-Photo.

On Cora, CiteSeer, and CoraFull, three GRACE pretraining seeds yield substantially different partitions (adjusted Rand index [42]: 0.247–0.365). After averaging over three downstream seeds, the standard deviations of accuracy across pretraining seeds are 0.13, 0.45, and 0.60 percentage points, respectively. GB-LCSS exceeds MLP+CE for every pretraining seed.

4.3.4. Sensitivity to construction and target parameters

Supplementary Appendix I.1 varies one grouping or voting parameter at a time on the five datasets under uniform 0.4 noise, with fixed representations and five paired seeds. The largest losses occur when a higher splitting threshold leaves all nodes in one ball or when a higher voting threshold sharply reduces the number of nodes receiving consensus supervision. The remaining tested settings stay within three percentage points of the default. A larger ball budget improves accuracy on CoraFull, and assigning more weight to the most-voted class improves accuracy on four datasets.

4.4. Graph-teacher extension and computational cost

4.4.1. Graph-teacher distillation

Table 8 tests whether GB-LCSS targets also improve a distilled MLP student (Section 3.5) under uniform 0.4 and 0.6. Direct MLP training uses labeled nodes only; distillation routes use teacher predictions but deploy the same two-layer MLP. The GB-LCSS GCN differs from GCN(X)+CE only by replacing observed one-hot labels with Eq. (12); other routes are detailed in Section 4.1.3 and Supplementary Appendix F.

Training the GCN teacher with GB-LCSS targets instead of the observed labels raises the mean accuracy of the distilled student in nine of the ten settings; the only exception is CoraFull under uniform 0.4. Target refinement can therefore improve the supervision transferred through a graph teacher. Among the five distillation routes, this

Table 8: Test accuracy of deployed MLPs under direct training and graph-to-MLP distillation.

Training route	**Cora**	**CiteSeer**	**CoraFull**	**Coauthor-CS**	**Amazon-Photo**
(a) Uniform 0.4					
GB-LCSS, direct MLP training	57.49±0.76	55.24±2.04	37.39±0.51	90.71±0.59	82.12±1.08
GCN(X)+CE teacher	77.95±2.62	66.09±1.38	**58.74±0.82**	93.20±0.09	89.86±0.77
FF-G2M	77.65±2.97	65.80±0.49	56.58±0.93	91.58±0.24	84.82±0.84
KRD	71.77±2.20	64.96±1.03	47.12±0.95	91.39±0.49	85.17±0.94
VQGraph	78.89±2.34	65.64±1.72	58.52±0.77	93.25±0.09	**90.54±0.85**
GCN teacher trained on GB-LCSS targets	**81.35±1.05**	**69.27±1.57**	57.29±0.24	**93.83±0.21**	90.40±0.77
(b) Uniform 0.6					
GB-LCSS, direct MLP training	41.86±2.30	42.02±3.14	27.43±0.89	87.19±0.69	71.45±3.75
GCN(X)+CE teacher	65.50±3.28	53.30±4.42	51.63±2.02	91.13±0.58	84.80±1.48
FF-G2M	63.63±2.16	52.32±4.20	45.78±2.90	86.50±1.01	73.08±2.82
KRD	61.28±1.17	54.17±3.14	41.61±2.43	89.80±0.29	80.30±0.51
VQGraph	65.75±2.72	53.34±4.47	52.01±0.65	90.71±1.24	84.77±2.19
GCN teacher trained on GB-LCSS targets	**71.34±3.91**	**58.93±4.62**	**52.32±1.26**	**92.40±0.65**	**86.81±1.94**

student attains the highest mean accuracy in eight of the ten settings; on CoraFull and Amazon-Photo under uniform 0.4, GCN(X)+CE and VQGraph are higher by 1.45 and 0.14 points, respectively. The distilled student exceeds direct MLP training in all ten settings, by 3.12–23.86 points under uniform 0.4 and 5.21–29.48 points under uniform 0.6. The extension adds a GCN teacher and distillation over all nodes during training, while retaining the same MLP architecture at inference.

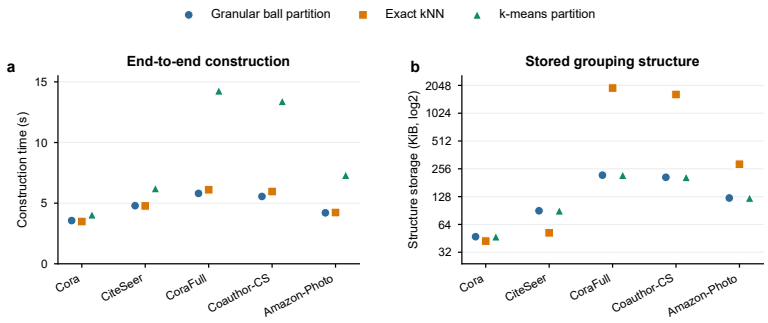


Figure 5: Construction cost of granular ball, exact-kNN, and k-means grouping. (a) End-to-end construction time. (b) Storage of the grouping structure. Both panels show seed-0 measurements.

4.4.2. Computational cost

Because GB-LCSS and MLP+CE deploy the same MLP, we report only additional training costs. Measurements use Windows 10 (build 19045), an AMD Radeon RX 7700 XT, Python 3.12.10, PyTorch 2.9.1+rocm7.2.1, and PyTorch Geometric 2.7.0. On Cora, CiteSeer, and CoraFull, GRACE pretraining takes 0.07, 0.13, and 74.48 minutes, respectively; peak allocated memory is 0.40, 0.67, and 12.73 GiB, respectively. Times are averaged over three pretraining seeds; memory is the largest `torch.cuda.max_memory_allocated()` peak. Each frozen representation is computed once per dataset and reused across downstream runs.

Fig. 5 compares the construction time and storage of the three grouping rules. The granular ball partition stores one class-count vector per ball, whereas exact kNN stores one neighbor list per training node. In the seed-0 measurements, granular ball grouping requires less storage than exact kNN on CoraFull, Coauthor-CS, and Amazon-Photo, with the largest differences on the two largest graphs. Its construction time is also the shortest among the three grouping rules on three of the five datasets.

4.5. Discussion

Consistent labels within a ball can offset isolated annotation errors, but the label-free representation determines which nodes provide that evidence. Random-grouping and label-smoothing controls support the importance of this choice beyond softening alone (Table 5). Leave-one-out voting also keeps the current annotation out of its

own consensus, as required by Corollary 1. Under uniform 0.4 noise, error removed from corrupted labels exceeds that introduced on correct labels (Supplementary Appendix I.2).

The usefulness of local evidence depends on both its class composition and the number of available votes. Proposition 1 relates these quantities to voting reliability under independent symmetric noise. Agreement alone is insufficient when the local labels favor an incorrect class, since the consensus can then move the target away from the true class. The representation and parameter studies show lower accuracy when local groups fail to form or voting support becomes sparse. Under uniform 0.4 noise, CoraFull has both the lowest proportion of training nodes with a positive clean-vote margin (36.13%) and the smallest net reduction in target Brier score among the five datasets (Supplementary Appendix I.2). With clean training labels, refinement raises mean accuracy on one dataset and lowers it on four, with all changes within 2.00 percentage points (Supplementary Appendix D). On the six additional datasets, GB-LCSS exceeds the strongest noisy-label baseline on three (Supplementary Appendix G). These results indicate that the benefit varies across datasets and supervision conditions.

GB-LCSS, Graph-MLP, and GLNN all deploy attribute-only models but use the training graph differently: Graph-MLP shapes representations, GLNN transfers teacher predictions, and GB-LCSS refines targets. Its advantage over Graph-MLP is dataset dependent (Table 4), while the distillation experiment shows that the same targets can also benefit an MLP student through a GCN teacher (Table 8).

Granular balls provide a practical organization for local-consensus supervision. Each node belongs to one ball, whose class counts are shared when constructing targets for its training nodes. Geometric splitting adapts the partition to the representation while limiting the number of balls and enforcing a minimum size for each child ball. Granular ball grouping requires less storage than exact kNN on the three larger graphs and has the shortest construction time on three datasets (Fig. 5). These results support its use as a grouping mechanism that combines competitive accuracy, shared regional statistics, and controlled construction costs.

5. Conclusion

We presented GB-LCSS for learning from noisy training labels when the deployed classifier receives only node attributes. The method organizes nodes into granular balls in a frozen, label-free graph representation and combines each observed label with the consensus of the other training nodes in its ball. Our analysis relates local-vote reliability to the number of available votes and their clean-class margin under symmetric noise, decomposes the squared error of the soft target, and gives a sufficient condition for it to have lower expected error than the observed label.

Across five datasets and five noise settings, GB-LCSS achieves the highest five-seed mean accuracy in 24 of the 25 settings, compared with seven noisy-label baselines that use the same deployed MLP architecture. Its soft targets have lower mean Brier scores than the observed one-hot labels in all 25 settings. The controlled comparisons support the importance of which training nodes contribute to the local consensus, with granular balls providing a compact way to share label statistics. The distillation experiments further show that these targets can improve the MLP student when they are used to train its graph teacher.

The present evaluation focuses on synthetic label noise in a fixed-graph transductive setting, with clean validation labels used for model selection. Future work will test the method on naturally occurring annotation errors and develop model-selection procedures for imperfect validation labels. Another direction is to evaluate generalization to newly arriving nodes and to investigate how the representations and granular ball partitions can be updated as the training graph grows.

Acknowledgements

This work was supported by the Yangtze River Delta Social Assistance Public Service Platform (B801065001FY050) and the National Natural Science Foundation of China (Nos. 62276136, 62206129, and 62375133).

References

[1] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations, 2017.

[2] Y. Hu, H. You, Z. Wang, Z. Wang, E. Zhou, Y. Gao, Graph-MLP: Node classification without message passing in graph (2021). arXiv:2106.04051.

[3] S. Zhang, Y. Liu, Y. Sun, N. Shah, Graph-less neural networks: Teaching old MLPs new tricks via distillation, in: International Conference on Learning Representations, 2022.

[4] Z. Wang, Z. Zhang, C. Zhang, Y. Ye, Training MLPs on graphs without supervision, in: Proceedings of the ACM International Conference on Web Search and Data Mining, 2025, pp. 697–706.

[5] L. Wu, H. Lin, Y. Huang, T. Fan, S. Z. Li, Extracting low-/high-frequency knowledge from graph neural networks and injecting it into MLPs: An effective GNN-to-MLP distillation framework, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 10351–10360.

[6] S. Liu, J. Niles-Weed, N. Razavian, C. Fernandez-Granda, Early-learning regularization prevents memorization of noisy labels, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 20331–20342.

[7] Z. Wang, D. Sun, S. Zhou, H. Wang, J. Fan, L. Huang, J. Bu, NoisyGL: A comprehensive benchmark for graph neural networks under label noise, in: Advances in Neural Information Processing Systems, Vol. 37, 2024, pp. 38142–38170.

[8] E. Dai, C. Aggarwal, S. Wang, NRGNN: Learning a label noise resistant graph neural network on sparsely and noisily labeled graphs, in: Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2021, pp. 227–236.

[9] S. Qian, H. Ying, R. Hu, J. Zhou, J. Chen, D. Z. Chen, J. Wu, Robust training of graph neural networks via noise governance, in: Proceedings of the ACM International Conference on Web Search and Data Mining, 2023, pp. 607–615.

[10] Y. Wu, J. Yao, X. Xia, J. Yu, R. Wang, B. Han, T. Liu, Mitigating label noise on graphs via topological sample selection, in: Proceedings of the International

Conference on Machine Learning, Vol. 235 of Proceedings of Machine Learning Research, 2024, pp. 53944–53972.

[11] A. Iscen, J. Valmadre, A. Arnab, C. Schmid, Learning with neighbor consistency for noisy labels, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 4672–4681.

[12] D. Bahri, H. Jiang, M. Gupta, Deep k-NN for noisy labels, in: Proceedings of the International Conference on Machine Learning, Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 540–550.

[13] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, L. Wang, Deep graph contrastive representation learning, in: ICML Workshop on Graph Representation Learning and Beyond, 2020.

[14] S. Xia, Y. Liu, X. Ding, G. Wang, H. Yu, Y. Luo, Granular ball computing classifiers for efficient, scalable and robust learning, Information Sciences 483 (2019) 136–152.

[15] S. Xia, S. Zheng, G. Wang, X. Gao, B. Wang, Granular ball sampling for noisy label classification or imbalanced classification, IEEE Transactions on Neural Networks and Learning Systems 34 (4) (2023) 2144–2155.

[16] S. Xia, X. Dai, G. Wang, X. Gao, E. Giem, An efficient and adaptive granular-ball generation method in classification problem, IEEE Transactions on Neural Networks and Learning Systems 35 (4) (2024) 5319–5331.

[17] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. W. Tsang, M. Sugiyama, Co-teaching: Robust training of deep neural networks with extremely noisy labels, in: Advances in Neural Information Processing Systems, Vol. 31, 2018, pp. 8527–8537.

[18] H. Wei, L. Feng, X. Chen, B. An, Combating noisy labels by agreement: A joint training method with co-regularization, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 13726–13735.

[19] Z. Zhou, R. Li, W. Ai, X. Li, Z. Teng, B. Zhang, J. Du, Affinity-aware uncertainty quantification for learning with noisy labels, Pattern Recognition 172 (2026) 112495. doi:10.1016/j.patcog.2025.112495.

[20] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, J. Bailey, Symmetric cross entropy for robust learning with noisy labels, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2019, pp. 322–330.

[21] H. Wei, H. Zhuang, R. Xie, L. Feng, G. Niu, B. An, Y. Li, Mitigating memorization of noisy labels by clipping the model prediction, in: Proceedings of the International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 36868–36886.

[22] Y. Zhu, L. Feng, Z. Deng, Y. Chen, R. Amor, M. Witbrock, Robust node classification on graph data with graph and label noise, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 38, 2024, pp. 17220–17227.

[23] Z. Liu, H. Yu, X. Luo, Mitigating label noise in graph learning with information bottleneck-guided aggregation and adversarial consistency, Pattern Recognition 180 (2026) 114271.

[24] Z. Fang, S. Du, Z. Wu, Z. Cai, Y. Tan, S. Wang, Z. Lin, Harnessing noisy LLM annotations: Confidence-calibrated node selection on text-attributed graphs, Pattern Recognition 180 (2026) 114379. doi:10.1016/j.patcog.2026.114379.

[25] L. Wu, J. Xia, H. Lin, Z. Gao, Z. Liu, G. Zhao, S. Z. Li, Teaching yourself: Graph self-distillation on neighborhood for node classification (2022). arXiv:2210.02097.

[26] L. Wu, H. Lin, Y. Huang, S. Z. Li, Quantifying the knowledge in GNNs for reliable distillation into MLPs, in: Proceedings of the International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 37571–37581.

[27] L. Yang, Y. Tian, M. Xu, Z. Liu, S. Hong, W. Qu, W. Zhang, B. Cui, M. Zhang, J. Leskovec, VQGraph: Rethinking graph representation space for bridging GNNs and MLPs, in: International Conference on Learning Representations, 2024.

[28] J. Heo, M. Park, D. Kim, Posterior label smoothing for node classification, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 40, 2026, pp. 21708–21716.

[29] K. Zhou, S.-H. Choi, Z. Liu, N. Liu, F. Yang, R. Chen, L. Li, X. Hu, Adaptive label smoothing to regularize large-scale graph training, in: Proceedings of the 2023 SIAM International Conference on Data Mining, 2023, pp. 55–63.

[30] Y. Wang, Y. Cai, Y. Liang, W. Wang, H. Ding, M. Chen, J. Tang, B. Hooi, Structure-aware label smoothing for graph neural networks (2021). arXiv:2112.00499.

[31] Y. Xie, Z. Xu, J. Zhang, Z. Wang, S. Ji, Self-supervised learning of graph neural networks: A unified review, IEEE Transactions on Pattern Analysis and Machine Intelligence 45 (2) (2023) 2412–2429.

[32] J. Xie, X. Xiang, S. Xia, L. Jiang, G. Wang, X. Gao, MGNR: A multi-granularity neighbor relationship and its application in KNN classification and clustering methods, IEEE Transactions on Pattern Analysis and Machine Intelligence 46 (12) (2024) 7956–7972.

[33] Y. Li, X. Ouyang, J. Zhang, C. Pan, L. Ren, X. Yang, Beneficial noise learning for open intent classification via granular-ball representation, Pattern Recognition 177 (2026) 113283. doi:10.1016/j.patcog.2026.113283.

[34] S. Xia, X. Ma, Z. Liu, C. Liu, S. Zhao, G. Wang, Graph coarsening via supervised granular-ball for scalable graph neural network training, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 39, 2025, pp. 12872–12880.

[35] S. Xia, G. Wang, G. Xu, S. Zhao, G. Wang, GBGC: Efficient and adaptive graph coarsening via granular-ball computing, in: Proceedings of the International Joint Conference on Artificial Intelligence, 2025, pp. 3489–3497.

[36] W. Hoeffding, Probability inequalities for sums of bounded random variables, Journal of the American Statistical Association 58 (301) (1963) 13–30.

[37] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad, Collective classification in network data, AI Magazine 29 (3) (2008) 93–106.

[38] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, in: Relational Representation Learning Workshop, NeurIPS, 2018.

[39] D. Arthur, S. Vassilvitskii, k-means++: The advantages of careful seeding, in:

Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms, 2007, pp. 1027–1035.

[40] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[41] G. W. Brier, Verification of forecasts expressed in terms of probability, Monthly Weather Review 78 (1) (1950) 1–3.

[42] L. Hubert, P. Arabie, Comparing partitions, Journal of Classification 2 (1) (1985) 193–218.